

Ur Sandhai

AI for Bharat, Prototype Round Write-Up

Live demo: <https://sruthisureshkumar-arch.github.io/ursandhai/>

Source code: <https://github.com/sruthisureshkumar-arch/ursandhai>

What This Prototype Demonstrates

Ur Sandhai lets farmers within about ten kilometres of each other share what they're selling, at what price, and how strong the demand feels, then turns those reports into a same-day recommendation: the best time to sell, the crop worth prioritising, and whether to hold back because too many neighbours are already offering the same thing. The recommendation is spoken and shown in both Tamil and English. This build is a single self-contained HTML file with no backend and no build step, by design, so it can be opened and tested in a few seconds without installing anything.

What's Actually Working Right Now

The geo-radius filtering is a real Haversine distance calculation, written in plain JavaScript, that keeps each report to a ten-kilometre radius and a seventy-two hour window. Prices are bucketed into two-hour windows and averaged to surface which hours are currently fetching the best prices. A weighted scoring formula combines average price, demand strength, and a supply-pressure penalty, so the app accounts for oversupply nearby rather than just recommending whichever crop happens to be selling for the most. The ten kilometre radius, the farmer's position, and every nearby report are rendered as a real vector map generated from the same coordinate math the scoring logic uses, not a static image. Every recommendation is produced in both English and Tamil from the same underlying data, and when fewer than five reports exist nearby, the app falls back to a baseline mandi price rather than guessing from thin data.

What's Mocked for This Round, and Why

Two pieces from the original proposal are stood in with free browser APIs for this demo, since wiring up billed cloud services for a prototype round didn't seem like the right place to spend the team's limited AWS free-tier hours. Voice input uses the browser's Web Speech API instead of Amazon Transcribe, listening in Tamil and transcribing locally, with the same regex-based parsing that would run against Transcribe's output extracting a price from the transcript. Spoken recommendations use the Web Speech Synthesis API instead of Amazon Polly, reading the Tamil recommendation aloud with a Tamil voice. Everything downstream of those two inputs, meaning the scoring, the map, and the bilingual text generation, is real and does not depend on any external service. Photo-based crop and price-board reading, and the next-day forecasting model, are not wired into this build yet; they're scoped as the next milestone rather than mocked, since the report form already asks for the data those pieces would extract.

Tools, APIs, and Datasets

Layer	Used now	Planned for production
Frontend	Plain HTML, CSS, and JavaScript, no framework or build tool	Same, or a lightweight framework if report volume grows
Design	Figma, used to prototype the visual language before implementing it in code	Same
Voice input	Web Speech API (SpeechRecognition), free and browser-native	Amazon Transcribe (Tamil and Hindi) within the AWS free tier
Text-to-speech	Web Speech Synthesis API, free and browser-native	Amazon Polly within the AWS free tier
Vision and OCR	Not yet implemented	Amazon Rekognition for crop identification, Textract for price board and slip reading
Forecasting	Weighted scoring formula, not a trained model	Prophet and scikit-learn, retrained nightly on crowd reports via a free GitHub Actions workflow
Government dataset	A small hard-coded baseline standing in for the real feed	Agmarknet mandi prices and arrivals, pulled from data.gov.in at no cost
Hosting	GitHub Pages, free static hosting	Same for the demo, plus a small backend once reports need to persist across visitors
Data storage	In-memory JavaScript array, resets on page reload	Supabase Postgres free tier, chosen for built-in geo queries
Fonts	Playfair Display and Inter, served from Google Fonts	Same

Honest Note on Scope

This is a working demonstration of the core idea, hyperlocal crowd data turned into a same-day, bilingual selling recommendation, not a finished production system. The parts that are mocked are mocked with free, zero-setup browser APIs specifically so the full flow can be tested without needing API keys or a backend. The GitHub repository's README carries the same real-versus-mocked breakdown alongside instructions for running the prototype locally.